

PARALELNI SISTEMI – nova akreditacija

1. a) Napisati MPI program koji realizuje množenje matrice $A_{m \times k}$ i vektora $\vec{b}_k$. Takođe, program kao rezultat treba da generiše i proizvod elemenata svake kolone matrice A . Matrica A i vektor b se inicijalizuju u master procesu. Matrica A je distribuirana u blokove po vrstama (p -broj procesa, k je deljivo sa p) i to tako da proces P_i dobija vrste sa indeksima l , $l \bmod p = i$ ($0 \leq i \leq p-1$), tj. vrste sa indeksima $i, i+p, i+2p, \dots, i+m-p$. Master proces distribuira odgovarajuće vrste matrice A (direktno iz matrice) i ceo vektor b svim procesima. Predvideti da se slanje svih vrsta matrice A jednom procesu obavlja odjednom. Predvideti da se slanje elementata vektora b jednom procesu obavlja odjednom. Svaki proces obavlja odgovarajuća izračunavanja i učestvuje u generisanju rezultata. Zadatak rešiti korišćenjem grupnih operacija.

- b) Napisati MPI program koji kreira komunikator koga čine svi procesi sa identifikatorima deljivim sa zadatom vrednošću x . Obezbediti da svaki proces ove grupe dobije po jednu kolonu matrice A . Odštampati identifikatore procesa koji pripadaju novom komunikatoru, kao i minimum svih suma primljenih kolona matrice A .

2. Na programskom jeziku C/C++, korišćenjem CUDA tehnologije napisati program koji u matrici A , koja predstavlja monohromatsku sliku noćnog neba, pronalazi piksele koji potencijalno predstavljaju zvezde. Matrica se sastoji od piksela koji imaju vrednosti između 0 i 255. Za svaki piksel koji ima vrednost veću od 128, treba proveriti da li je prosečna vrednost njegovih susednih piksela (u okruženju od po jednog piksela levo, gore, desno, dole i dijagonalno), veća od 64. Ukoliko jeste, na poziciju piksela u rezultujućoj matrici B upisati 1, a ukoliko nije, 0. Obratiti pažnju na efikasnost paralelizacije. Omogućiti pozivanje kernela za matrice proizvoljne veličine.

3. Data je petlja (smatrati da je pre petlje inicijalizovan niz b od 9 elemenata):

```
int a = 6;
for (i = 0; i < N; i++)
{
    if (i >= N / 2)
        b[i] = b[i - 1];

    b[i]++;
    a *= 4;
}
```

- a) Da li je moguće izvršiti paralelizaciju date petlje bez modifikacija? Obrazložiti.
b) Ukoliko postoje zavisnosti, objasniti ih i jasno napisati o kom je tipu zavisnosti reč.
c) Transformisati petlju tako da paralelizacija bude moguća.
d) Razvijanjem petlje za konkretne inicijalizacije vrednosti a , N , i elemenata niza b , testirati sekvencijalno i paralelno rešenje i pokazati da je petlja korektno paralelizovana rešenjem pod c).

Napomena: zadaci koji daju odgovor samo na stavku c) neće biti bodovani.

4. a) U sledećem primeru identifikovati sve ZIV, SIV i MIV subscripte.

```
for j = 1, 10
for i = 1, 99
{
    A(i,j) = B(j) + X;
    B(j+1) = A(100-i,j-1) + Y;
}
```

- b) Odrediti sve vektore i pravce zavisnosti. Da li se unutrašnja petlja može vektorizovati?

5. U ESC (Extra Stage Cube) višestepenoj sprežnoj mreži koja povezuje 32 PE potrebno je poslati poruku iz čvora sa adresom 18 u čvorove sa adresma 4, 6, 12 i 14 (obaviti emisiju). Kako izgleda zaglavlje poruke za primarni i sekundarni put?

6. Korišćenjem Bernsteinovih uslova odrediti maksimalni paralelizam izneđu instrukcija u sledećem segmentu

S1: $X = Y + Z$

S2: $Z = U + V$

S3: $R = S + V$

S4: $Z = X + R$

S5: $Q = M + Z$

Ako se svaka od instrukcija izvršava za 1 vremensku jedinicu, za koliko vremena će se izvršiti paralelna verzija programa? Koliko je ubrzanje u tom slučaju? Koliko se maksimalno brzanje može postići paralelnim izvršenjem navedenog kodnog segmenta?

7. a) Šta od sledećeg nije tačno:

- OpenMP je API koji omogućava korišćenje multi-thread paralelizma
- Osnovne komponente OpenMP su kompajlerske directive, runtime biblioteka, i promenljive okruženja
- OpenMP implementacija postoji za Microsoft Windows platformu.
- OpenMP je namenjen za sisteme sa distribuiranom memorijom i garantuje efikasno korišćenje memorije
- OpenMP podržava UMA i NUMA arhitekture

b) Šta od navedenog nije moguće u OpenMP:

- zavisnosti po podacima u #pragma omp for
- konflikti po podacima u #pragma omp critical
- trka podataka u #pragma omp parallel
- Deadlock u #pragma omp parallel

c) Koja od navedenih OpenMP direktiva je pogrešno iskorišćena:

- #pragma omp for nowait
- #pragma omp parallel private(threadID)
- #pragma omp atomic(threadID)
- #pragma omp parallel for if (val)

8. U multiprocesorskom sistemu se za održavanje keš koherencije koristi Write once protokol. Inicijalno je keš memorija procesora P1 prazna. Popuniti donju tabelu koja prikazuje promene stanja keš bloka u procesoru P1 u zavisnosti od komandi koje su izdaju.

Događaj/komanda	Stanje keš bloka u P1	Akcija keš kontrolera P1 (napisati ništa ako nema akcije)
P_Read A		
P_Write A		
Read_Bl k A		
Write_Inv A		
P_Write A		
P_Read A		
Read_Bl k A		
P_write A		
P_write A		